

HomeService OS

Easy Phase-Wise Development Plan for 4 Members

Technology: HTML + CSS + JavaScript + Python Flask + MySQL + MySQL Workbench

Important: Follow the phases in this order. Build the core product first; add the AI Agent only later.

Member 1 & Member 3 handle backend/Python/database work. Member 2 & Member 4 handle frontend HTML/CSS/JS work.

Quick View

Phase	What we do
1	Requirements & Planning
2	System Design
3	Database Setup
4	Backend Setup
5	Frontend/UI
6	Login & Roles
7	Homes & Appliances
8	Services & Technicians
9	Service Request & Booking
10	Nearby Technician Matching
11	Move Mode
12	Service History & Maintenance
13	Payments, Reviews & Notifications
14	Statistics & Analytics
15	Integration
16	Testing & Bug Fixing
17	Deployment
18	AI Agent — Later

PHASE 1 — Requirements & Planning

- **Member 1:** List customer, home, appliance and Move Mode requirements, and services/technicians requirements.
- **Member 3:** List admin, payment, review, notification requirements, and booking/matching requirements.
- **Member 2:** List UI/UX requirements for services, technician, booking, payments and reviews screens.
- **Member 4:** List UI/UX requirements for customer, home, appliance, Move Mode, admin and statistics screens.
- **All 4:** Finalize project objective, features, technology and GitHub repository.
- **Output:** Complete requirements document.

PHASE 2 — System Design

- **Member 1:** Design Customer, Home, Appliance, Move Mode flows, and Services/Technician flows.
- **Member 3:** Design Admin, Payment, Review, Notification flows, and Booking/Matching flows.
- **Member 2:** Create wireframes and page list for services, technician, booking, payments and reviews screens.

- **Member 4:** Create wireframes and page list for customer, home, appliance, Move Mode, admin and statistics screens.
- **All 4:** Create use-case diagram, ER diagram, system architecture and page list.
- **Output:** Complete system design.

PHASE 3 — Database Setup

- **Member 1:** customers, homes, appliances, move_requests, move_appliances, technicians, services, technician_skills, technician_availability.
- **Member 3:** users, payments, reviews, complaints, notifications, maintenance_reminders, service_requests, bookings, service_history.
- **Member 2:** Review the fields needed by services/technician/booking/payments/reviews screens against the schema.
- **Member 4:** Review the fields needed by customer/home/appliance/admin/statistics screens against the schema.
- **All 4:** Create MySQL database, keys, relationships and test data.
- **Output:** Working MySQL database.

PHASE 4 — Backend Setup

- **Member 1:** Customer/Home/Appliance Flask routes, and Technician/Services Flask routes.
- **Member 3:** Admin/Analytics Flask routes, and Booking/Matching Flask routes.
- **Member 2:** Set up the frontend structure (static/css, static/js) for services, technician, booking, payments and reviews pages.
- **Member 4:** Set up the overall frontend project structure (templates, base layout) and customer/home/appliance/admin pages.
- **All 4:** Set up Python, Flask, MySQL connection, Git and project structure.
- **Output:** Flask successfully connected to MySQL.

PHASE 5 — Frontend / UI

- **Member 1:** Provide page content and data fields for customer, home, appliance pages, and services/technician pages.
- **Member 3:** Provide page content and data fields for admin, payments pages, and booking/matching pages.
- **Member 2:** Build the services, technician, booking, payments and reviews UI.
- **Member 4:** Build the design system (colors, fonts, buttons, cards, forms, navbar, sidebar) and customer, home, appliance, Move Mode, admin, statistics UI.
- **All 4:** Keep the same colors, fonts, navbar, sidebar, buttons, cards and forms.
- **Output:** Complete basic UI.

PHASE 6 — Login & Roles

- **Member 1:** Customer registration, login and profile, and technician registration, login and profile.
- **Member 3:** Admin login and authentication.
- **Member 2:** Login/registration UI for customer and technician.
- **Member 4:** Admin login UI and the three role-based dashboard layouts.
- **All 4:** Password hashing, sessions, logout and role-based access.
- **Output:** Customer / Technician / Admin dashboards work correctly.

PHASE 7 — Home & Appliance Management

- **Member 1:** Add/edit/delete/view homes; set current home; add/edit/delete/view appliances; warranty details; link appliances to services.
- **Member 3:** Connect appliance data to admin reports, maintenance data and booking creation.
- **Member 2:** Build the 'select appliance' step inside the service request/booking flow.

- **Member 4:** Build the home list/detail UI, appliance cards and add/edit forms with warranty fields.
- **Output:** Customer has a digital home and appliance profile.

PHASE 8 — Services & Technicians

- **Member 1:** Build service categories, prices, required skills, technician profiles, skills, experience, service area and availability.
- **Member 3:** Admin service management and technician verification.
- **Member 2:** Build the service catalog UI, service detail cards and technician profile/listing UI.
- **Member 4:** Build the admin technician-verification screen UI.
- **Output:** Working service and technician system.

PHASE 9 — Service Request & Booking

- **Member 1:** Customer selects home → appliance → service → problem → date/time → submit request.
- **Member 3:** Technician receives request, accepts/rejects, schedules and updates status; admin monitors bookings and cancellations.
- **Member 2:** Build the booking request wizard UI and the status-tracker UI.
- **Member 4:** Build the admin booking-monitoring dashboard UI.
- **Output:** Customer can request and book a service.

PHASE 10 — Nearby Technician Matching

- **Member 1:** Provide customer location and technician list data.
- **Member 3:** Match by location, distance, required skill, availability and rating; matching statistics and location-wise demand analysis.
- **Member 2:** Build the nearby-technician list/map UI with distance and rating badges.
- **Member 4:** Build the location-wise demand chart on the admin/statistics dashboard.
- **Output:** Customer can find suitable nearby technicians.

PHASE 11 — MOVE MODE

- **Member 1:** New home address and appliance selection.
- **Member 3:** Convert selected appliances into required services and find technicians near the NEW home; cost estimate, move summary and admin monitoring.
- **Member 2:** Build the required-services and nearby-technician-search steps of the Move Mode UI.
- **Member 4:** Build the new-address entry, appliance-selection and cost-estimate/booking-confirmation steps of the Move Mode UI.
- **All 4:** Test the full flow: Old Home → New Home → Appliances → Services → Nearby Technicians → Booking → Installation.
- **Output:** Complete Move-to-New-Home feature.

PHASE 12 — Service History & Maintenance

- **Member 1:** Customer service and appliance history.
- **Member 3:** Technician adds completed work and service details; maintenance reminders and service-frequency statistics.
- **Member 2:** Build the technician 'add completed work' UI form.
- **Member 4:** Build the customer service/appliance history timeline UI and maintenance-reminder UI.
- **Output:** Every appliance has useful service history.

PHASE 13 — Payments, Reviews & Notifications

- **Member 1:** Customer payment history, reviews and notifications.
- **Member 3:** Payment records, reviews, ratings, notifications and admin monitoring.
- **Member 2:** Build the payment summary UI, review/rating UI and notification center UI.
- **Member 4:** Build the admin monitoring UI for payments, reviews and notifications.

- **Output:** Complete service transaction flow.

PHASE 14 — Statistics & Analytics

- **Member 1:** Customer spending, service count, average cost and monthly expenses.
- **Member 3:** Technician jobs, rating, service time, cancellations, earnings, and admin customers, technicians, bookings, revenue, popular services and location demand.
- **Member 2:** Build statistics cards, charts, tables and filters for services/technician/booking-related views.
- **Member 4:** Build statistics cards, charts, tables and filters for customer and admin dashboards.
- **All 4:** Use Python/Pandas/NumPy/SciPy for suitable statistical analysis.
- **Output:** Customer, Technician and Admin analytics dashboards.

PHASE 15 — Complete Integration

- **Member 1:** Test Customer → Home → Appliance → Service → Booking → Move Mode.
- **Member 3:** Test Service → Technician → Matching → Booking → Completion, and Booking → Payment → Review → Notification → Statistics.
- **Member 2:** Connect services/technician/booking/payments/reviews UI to APIs and fix UI-side issues.
- **Member 4:** Connect customer/home/appliance/Move Mode/admin/statistics UI to APIs and fix UI-side issues.
- **All 4:** Connect every module and fix integration problems.
- **Output:** One complete working HomeService OS.

PHASE 16 — Testing & Bug Fixing

- **Member 1:** Test customer, home, appliance, Move Mode, services and technician backend logic.
- **Member 3:** Test admin, payments, reviews, statistics, booking and matching backend logic.
- **Member 2:** Test services/technician/booking/payments/reviews UI for responsiveness and UX.
- **Member 4:** Test customer/home/appliance/Move Mode/admin/statistics UI for responsiveness and UX.
- **All 4:** Test login, security, database, errors, responsiveness and complete workflows.
- **Output:** Stable application.

PHASE 17 — Deployment

- **Member 1:** Deploy customer/home/appliance and services/technician backend routes.
- **Member 3:** Deploy admin/booking/matching/statistics backend and production MySQL.
- **Member 2:** Optimize and deploy static assets for services/technician/booking/payments/reviews screens.
- **Member 4:** Optimize and deploy static assets for customer/home/appliance/Move Mode/admin/statistics screens.
- **All 4:** Environment variables, security checks and final production test.
- **Output:** Live HomeService OS.

PHASE 18 — AI AGENT — LATER

- **Member 1:** AI tools for customer/home/appliance data access, and technician/service tools (find technicians, check availability).
- **Member 3:** AI tools for admin/booking/payment/review data access and creating bookings.
- **Member 2:** Design and build the AI chat interface UI for services/technician/booking/payments/reviews interactions.
- **Member 4:** Design and build the overall AI chat interface UI and conversation UX for the customer-facing experience.
- **All 4:** Connect the AI agent to controlled backend functions instead of directly changing the database. Example: 'I moved to a new house and need my AC installed' → AI understands → finds service → finds nearby technicians → asks confirmation → creates booking.
- **Output:** AI-powered HomeService OS.

Final Order to Remember

Requirements → Design → Database → Backend → UI → Login → Home & Appliances → Services & Technicians → Service Request & Booking → Nearby Technician Matching → Move Mode → Service History & Maintenance → Payments/Reviews/Notifications → Statistics → Integration → Testing → Deployment → AI Agent later.

Team rule: Member 1 and Member 3 handle all backend/Python/database work between them. Member 2 and Member 4 handle all frontend HTML/CSS/JS work between them. During Phases 1–6, everyone builds the foundation together. From Phase 7 onward, each pair leads their side while helping with integration. Phases 15–17 are completed together.